MULTIMEDI

SPRING / SPRING BOOT TRAINING TRACK

Day 2 – Practice exercises

Six exercises: four on the topics of Day 2, and two that go further

PRACTICE

How the practice works

Six exercises

Four exercises follow the topics of Day 2, in three steps each. Two go further than the slides. Every step has a test: it is red at the start and turns green when you have fixed the problem.

Work in order

Later steps build on the earlier ones. Read what the test says, change the code, run it again, and look at the console.

Read the console

Every test prints where it starts and what was stored. SQL logging and the transaction log are on, so you can see what really happens.

Where you start and where you end

21 tests: 19 are red and 2 are green at the start (the green ones must stay green). When you are done, all 21 are green. Search the code for TODO step to find the places to change. Exercise 6 has no steps: it is an assignment.

Run one exercise: mvn test -Dtest=PersistencePracticeTest (the other test classes: Transactions-, Consistency-, Messaging-, Booking- and ConfirmationPracticeTest)



PRACTICE

The six exercises

1 Persistence context

ProductService. One transaction or not, dirty checking without save(), and readOnly = true that silently stops the flush.

2 Transactions

TransferService. A checked exception that does not roll back, REQUIRES_NEW that is ignored, and a swallowed exception in a batch.

3 Data consistency

Article and StockService. Optimistic locking with @Version, check-then-act on the last item, and a bulk update that bypasses the context.

4 Asynchronous integration

PaymentService. Send after the commit, the outbox for a broker that is down, and an idempotent consumer for duplicates.

5 Retry on a conflict

Beyond the slides, with guidance. Write a booking, detect a second customer with @Version, and write a retry loop around it.

6 Confirm only what is committed

Beyond the slides, on your own. An assignment with four requirements and no steps. Work it out yourself: the tests are the judge.

EXERCISE 1 – STARTER CODE

One persistence context per transaction

PRODUCTSERVICE.JAVA (STARTER)

```java
@Service
public class ProductService {
  private final ProductRepository repository;   // constructor omitted
  // TODO step 1: first and second must be the SAME object
  public boolean loadsTwiceGiveTheSameObject(Long id) {
    Product first = repository.findById(id).orElseThrow();
    Product second = repository.findById(id).orElseThrow();
    return first == second;
  }

  @Transactional
  public void applyDiscount(Long id, long percent) {
    // TODO step 2: lower the price, WITHOUT calling save()
  }

  @Transactional(readOnly = true)    // TODO step 3
  public void raiseAllPrices(long percent) {
    for (Product p : repository.findAll()) {
      p.setPrice(p.getPrice() * (100 + percent) / 100);
    }
  }
}
```

ProductRepository is a plain JpaRepository<Product, Long>. Before every test a laptop (1000) and a phone (500) are inserted. The test has no @Transactional. SQL logging is on.



YOUR TASK – EXERCISE 1

See the persistence context at work

1. Run step 1. loadsTwiceGiveTheSameObject() returns false. The method has no annotation. Add the one thing that lets both findById() calls share a persistence context, then run it again

2. Implement applyDiscount(): load the product and lower the price by the percentage, and do NOT call save(). The test reads the price from the database. Where does the update in the console come from?

3. Run step 3. raiseAllPrices() loads and changes every product, yet nothing is stored and there is no error. Find out why, fix it with the smallest change, and explain it to your neighbour

Run it: mvn test -Dtest=PersistencePracticeTest (3 red at the start, 3 green at the end)

EXERCISE 2 – STARTER CODE

Rollback rules and transaction boundaries

TRANSFERSERVICE.JAVA (STARTER)

```java
@Service
public class TransferService {
  // accounts, attemptRepository and attempts (an AttemptLogger bean) are injected

  @Transactional    // TODO step 1
  public void transfer(Long fromId, Long toId, long amount) throws InsufficientFundsException {
    logAttempt("transfer " + amount + " to " + toId);    // TODO step 2
    Account from = find(fromId);
    Account to = find(toId);
    from.setBalance(from.getBalance() - amount);
    to.setBalance(to.getBalance() + amount);
    if (from.getBalance() < 0) {
      throw new InsufficientFundsException("Not enough money");
    }
  }

  @Transactional(propagation = Propagation.REQUIRES_NEW)
  public void logAttempt(String description) {
    attemptRepository.save(new AttemptEntry(description));
  }
}
// BatchTransferService.transferAll() has @Transactional: TODO step 3
```

Before every test Anna has 100 and Ben has 0. AttemptLogger writes a row in the attempts table. InsufficientFundsException is a checked exception (it extends Exception).



YOUR TASK – EXERCISE 2

Make a failed transfer harmless

1. Anna (100) transfers 150 to Ben: InsufficientFundsException, yet both balances are stored. Look at the JpaTransactionManager log: commit or rollback? Make a failed transfer roll back completely. Do not move the check: fix it with the transaction settings

2. Every transfer must leave a row in the attempts table, also a failed one. logAttempt() has REQUIRES_NEW, yet the row disappears. Why? Fix it so the row survives (use the AttemptLogger bean, it has a TODO too)

3. transferAll() runs several transfers and swallows the failed ones, and you get an UnexpectedRollbackException. Read the log, then make a failed transfer harmless for the others

Run it: mvn test -Dtest=TransactionsPracticeTest (3 red at the start, 3 green at the end)

EXERCISE 3 – STARTER CODE

Let the database decide

ARTICLE, STOCKSERVICE, ARTICLEREPOSITORY (STARTER)

```
// Article.java
@Entity
public class Article {
  // TODO step 1: add a version field
  @Id @GeneratedValue private Long id;
  private String name;
  private long price;
}

// StockService.java
@Transactional
public void reserve(Long id, int amount) {
  StockItem item = items.findById(id).orElseThrow();
  if (item.getQuantity() < amount) throw new OutOfStockException(id);
  pretendToBeBusy();   // keep this line
  item.setQuantity(item.getQuantity() - amount);   // TODO step 2
}

// ArticleRepository.java
@Modifying   // TODO step 3
@Query("update Article a set a.price = a.price * 2")
int doublePrices();
```

Before every test the tables are empty. The test for step 2 simulates a second order that sneaks in between your check and your update. PriceService and the other repositories are ready.



YOUR TASK – EXERCISE 3

Three consistency problems

1. Anna and Ben load the same article, each changes a different field and both save. The second save silently overwrites the first. Make the second save fail loudly, with one extra field on Article

2. Two orders for the last ticket. Replace check-then-act by ONE statement: in StockItemRepository add a bulk update that subtracts the amount where the id matches and the quantity is enough, and check how many rows it changed. Keep pretendToBeBusy() before your update

3. doublePricesAndReload() runs a bulk update and then loads the article again, but the old price comes back. Change one annotation attribute so that the persistence context is cleared after the bulk update

Run it: mvn test -Dtest=ConsistencyPracticeTest (3 red at the start, 3 green at the end)

EXERCISE 4 – STARTER CODE

No shared transaction: send after the commit

PAYMENTSERVICE.JAVA (STARTER)

```java
@Service
public class PaymentService {
  // payments, broker, events and outbox are injected

  @Transactional
  public void register(Payment payment) {
    payments.save(payment);
    broker.send("payments", payment.getId());    // TODO step 1 and 2
    if (payment.getAmount() <= 0) {
      throw new IllegalArgumentException("A payment must be positive");
    }
  }
}

// PaymentMessagePublisher.on(PaymentReceived event)          TODO step 1
// OutboxRelay.publishPending()                               TODO step 2
// PaymentConsumer.handle(String messageId, Long paymentId)   TODO step 3
```

The broker is a fake that records what it sends, and the test can switch it off. OutboxEntry, OutboxRepository, LedgerEntry and ProcessedMessage are ready. The test calls OutboxRelay.publishPending() by hand.



YOUR TASK – EXERCISE 4

Make the messages reliable

1. A payment that is rolled back still sends a message. Publish a PaymentReceived event instead of calling the broker, and let PaymentMessagePublisher react only after the commit

2. The broker is down at the moment of the commit, and the message is gone. Save an OutboxEntry in the same transaction instead of publishing the event (remove the event of step 1), and let OutboxRelay send the waiting entries and mark each one as sent

3. A broker may deliver a message twice. Make PaymentConsumer.handle() safe for duplicates with the ProcessedMessage table, so that the ledger gets one entry for each message

Run it: mvn test -Dtest=MessagingPracticeTest (3 red and 1 green at the start, 4 green at the end)

EXERCISE 5 – STARTER CODE

Book a seat, and try again

BOOKINGTRANSACTION, BOOKINGSERVICE (STARTER)

```
@Service
public class BookingTransaction {
  // flights (a FlightRepository) is injected

  @Transactional
  public void bookSeat(Long flightId) {
    // TODO step 1: load the flight
    // TODO step 1: throw a SoldOutException when no seats are left
    pretendToBeBusy();   // keep this line: the test simulates a second customer here
    // TODO step 1: take one seat
  }
}

@Service   // not transactional, on purpose
public class BookingService {
  public void bookWithRetry(Long flightId) {
    // TODO step 3: call bookSeat(), and try again on a conflict
  }
}

// Flight.java:   // TODO step 2: add a version field
```

A Flight has a code and a number of seats left. SoldOutException, FlightRepository and the test are ready. In the test a second customer books while the first booking is still busy.



YOUR TASK – EXERCISE 5

Book a seat, and try again

1. Write bookSeat(): load the flight, throw a SoldOutException when no seats are left, and take one seat. The skeleton shows where each part goes. Run step 1: you can book until the flight is sold out

2. Run step 2. A second customer books while the first is busy. No exception, and a seat is lost. Add the missing field to Flight, so that the slower booking fails with an OptimisticLockingFailureException

3. Write bookWithRetry(): call bookSeat() and try again on a conflict, at most 3 attempts. Retry only on a conflict: a sold out flight must still throw. Why is this method in another bean than bookSeat()?

Run it: mvn test -Dtest=BookingPracticeTest (4 red at the start, 4 green at the end)

EXERCISE 6 – ASSIGNMENT

Confirm only what is committed

1 A valid order

register(reference, amount) stores a PurchaseOrder. After the commit a confirmation mail goes out (FakeMailSender) and a SentMail row is stored for the audit.

2 Over the limit

An amount above 1000 throws a LimitExceededException (a checked exception). Nothing remains: no order, no mail and no SentMail row.

3 A failing commit

The same reference twice breaks a unique constraint at commit. A mail cannot be taken back: no second mail may go out.

4 A caller that rolls back

register() may run inside a bigger transaction of the caller. If that rolls back, nothing remains and no mail goes out.

What is given, and what is not

OrderRegistrationService with an empty register(), the entities, the repositories and FakeMailSender. Everything else you create yourself. There are no steps and no hints: the four tests tell you what is wrong, and the console tells you more.

Run it: mvn test -Dtest=ConfirmationPracticeTest (3 red and 1 green at the start, 4 green at the end)



RECAP – PRACTICE

What you just fixed

- **Persistence context: one per transaction**
  Without @Transactional there is no shared context and no dirty checking. With readOnly = true there is no flush, and no error

- **Transactions: rollback rules and boundaries**
  A checked exception commits by default, REQUIRES_NEW needs another bean, and a swallowed failure makes the outer transaction rollback-only

- **Consistency: let the database decide**
  @Version turns a lost update into an error and a retry loop recovers from it, one conditional update beats check-then-act, and a bulk update bypasses the persistence context

- **Messaging: no shared transaction**
  Send after the commit, use an outbox so that a message is never lost, make the consumer idempotent, and give database work in an after-commit listener its own transaction